

EMPLOYEE MANAGEMENT SYSTEM

Database Fundamentals — Capstone Project

PL/SQL · Oracle Database

Covers: ERD · Mapping · Normalization · SQL DML · Aggregation · Joins · Subqueries ·
Views · Stored Procedures · Triggers · Scheduler Jobs

Difficulty	Points	Color
Easy	1 pt each	Green
Medium	2 pts each	Orange
Hard	3 pts	Red

Project Scoring Summary

Section	Tasks	Max Points
ERD Design & Mapping	3	5
Normalization	3	5
SQL DML	5	7
Aggregation Functions	5	7
Joins	5	7
Subqueries	5	7
Views	5	7
Stored Procedures	5	7
Triggers	5	7
Scheduler Jobs	5	7
TOTAL	46 tasks	67 points

Project Overview

This capstone project tests your command of Oracle PL/SQL on a realistic Employee Management System. The database tracks employees, departments, salary information, payroll calculations, employee qualifications, and leave management. You will design the schema, manipulate data, build analytical queries, and automate business logic using Oracle-specific features.

Entity Reference

The six entities of this system are listed below with their attributes and primary keys only. Foreign keys are intentionally omitted — you will identify them yourself in Section 01.

Entity	Primary Key	Attributes
JOB_DEPARTMENT	job_ID	job_dept, name, description, salary_range
SALARY_BONUS	salary_ID	amount, annual, bonus
QUALIFICATION	qual_ID	position, requirements, date_in
EMPLOYEE	emp_ID	fname, lname, gender, age, contact_add, emp_email, emp_pass
PAYROLL	payroll_ID	date, report, total_amount
LEAVE	leave_ID	date, reason

Submission Rules: (1) All SQL must run on Oracle 19c or later. (2) Use meaningful aliases and comments in your scripts. (3) Each section must be submitted as a separate .sql file named after the section (e.g., *03_normalization.sql*). (4) Screenshots of output are required for Trigger and Scheduler Job tasks.

01 ERD Design & Mapping

Using only the entity names, attributes, and relationship descriptions below, you will design the full ERD from scratch and then map it to a relational schema. No foreign keys are given — you must identify them yourself.

Relationship Descriptions

- One JOB_DEPARTMENT can have many SALARY_BONUS records. Each SALARY_BONUS belongs to one department.
- One JOB_DEPARTMENT can have many EMPLOYEEs. Each EMPLOYEE belongs to one department.
- Each EMPLOYEE is linked to exactly one SALARY_BONUS record.
- One EMPLOYEE can have many QUALIFICATION records. Each QUALIFICATION belongs to one employee.
- One EMPLOYEE can have many LEAVE records. Each LEAVE record belongs to one employee.
- One EMPLOYEE can appear in many PAYROLL records. Each PAYROLL record belongs to one employee.
- One JOB_DEPARTMENT can be referenced in many PAYROLL records.
- One SALARY_BONUS record can be referenced in many PAYROLL records.
- One LEAVE record can be referenced in a PAYROLL record (optional link).

Task 1: Draw the Full ERD

Easy

1 pts

Using the entity reference on the cover page and the relationship descriptions above, draw the complete ERD using **Crow's Foot (IE) notation**. Your diagram must include:

- All 6 entities as rectangles with all their attributes listed inside.
- The Primary Key of each entity clearly underlined.
- All 9 relationships drawn as lines with correct cardinality symbols at both ends (one: single bar, many: three-pronged fork).
- Participation constraints on both sides of each relationship (mandatory: double bar, optional: small circle).
- Relationship labels (verbs) on each connecting line.
- If any entity qualifies as a weak entity, mark it with a double rectangle.

■ Use draw.io, Lucidchart, Microsoft Visio, or hand-draw and scan. Submit as PDF or image file.

Task 2: Map the ERD to a Relational Schema

Easy

1 pts

Convert your ERD into a written relational schema by applying the standard mapping rules:

- **Rule 1:** Every strong entity becomes a table. Its PK stays the PK.
- **Rule 2:** For every 1:N relationship, place the PK of the 'One' side as a FK in the 'Many' side table.
- **Rule 3:** For 1:1 relationships, place the FK on the total-participation (mandatory) side.
- **Rule 4:** Any M:N relationship requires a new junction table with both PKs as a composite PK.

Write each table using this format:

```
TABLE_NAME ( PK_column , attr1 , attr2 , FK_column )
```

For each FK, explicitly state the table and column it references, e.g.:

```
PAYROLL.emp_ID references EMPLOYEE(emp_ID)
```

Count and list all foreign keys in a summary at the end.

Task 3: Write the Full DDL CREATE Statements

Medium

2 pts

Based on your mapped schema, write Oracle DDL to create all six tables:

- Use appropriate Oracle data types: VARCHAR2, NUMBER, DATE — with suitable sizes.
- Declare PRIMARY KEY, FOREIGN KEY, NOT NULL constraints for all tables.
- Create tables in the correct dependency order so all FK references are satisfied.
- Add a CHECK constraint on EMPLOYEE.gender allowing only 'M' or 'F'.
- Add a CHECK constraint on SALARY_BONUS.amount ensuring it is greater than 0.
- Create one Oracle SEQUENCE per table to auto-generate primary key values.

■ Tables with foreign keys must be created after the tables they reference.

02 Normalization

Task 1: First Normal Form (1NF) Analysis

Easy

1 pts

Given the following un-normalized flat table that was used before the EMS database was designed:

```
EMP_RAW ( emp_id, full_name, phone_numbers, dept_name, dept_location, job_titles, salary )
```

- (a) Identify every violation of 1NF in this table and explain each one clearly.
- (b) Rewrite the table in 1NF. Show the resulting table(s) with at least 3 rows of sample data.
- (c) Check your own EMS schema (from Section 01) — confirm that all 6 tables satisfy 1NF and explain why.

Task 2: Second Normal Form (2NF) Analysis

Easy

1 pts

Given the following table (already in 1NF) with a composite primary key (emp_ID, job_ID):

```
PAYROLL_RAW ( emp_ID, job_ID, emp_name, job_title, salary_range, pay_date, total_amount )
```

- (a) Identify all partial dependencies — attributes that depend on only part of the composite key.
- (b) Decompose the table into 2NF relations. Write the schema for each resulting table.
- (c) Review your EMS schema from Section 01 — does any table have a composite PK with partial dependencies? Justify your answer.

Task 3: Third Normal Form (3NF) Analysis

Medium

2 pts

Given the following table (already in 2NF):

```
EMPLOYEE_EXT ( emp_ID, dept_ID, dept_name, dept_manager, manager_phone, emp_salary )
```

- (a) Identify all transitive dependencies — non-key attributes that determine other non-key attributes.
- (b) Decompose into 3NF. Provide the final schema and show that no transitive dependencies remain.
- (c) Analyze your full EMS schema from Section 01 against 3NF. For each table, confirm no transitive dependencies exist, or fix any that you find.
- (d) Write a final statement confirming your complete schema is in 3NF.

03 SQL DML — INSERT / UPDATE / DELETE / SELECT

Task 1: Populate All Tables with Seed Data

Easy

1 pts

Write INSERT statements to populate the database with realistic seed data:

- At least **5 departments**, **10 employees**, **5 salary/bonus records**, **8 payroll records**, **5 leave records**, **5 qualification records**.
- Use Oracle sequences (CREATE SEQUENCE ... START WITH 1 INCREMENT BY 1) to generate PKs for all tables.
- Ensure referential integrity — do not insert child records before parent records.

■ *Tip: wrap all inserts in a transaction and use COMMIT at the end.*

Task 2: Conditional SELECT Queries

Easy

1 pts

Write SELECT queries for each of the following requirements:

- List all employees whose age is between 25 and 40, ordered by last name ascending.
- Retrieve all payroll records where total_amount exceeds 5000, showing employee name and department.
- Find all employees who have taken leave with reason containing the word 'sick' (case-insensitive).
- List all departments that have no employees assigned. (Use outer join or NOT EXISTS.)

Task 3: Bulk UPDATE Scenarios

Medium

2 pts

Write UPDATE statements for the following business scenarios:

- Give a 10% salary increase to all employees in the 'Engineering' department.
- Update the emp_email of all employees to lowercase using Oracle's LOWER() function.
- Set the salary_range in JOB_DEPARTMENT to 'REVISED' for any department whose average total payroll exceeds 8000.

■ *Use a subquery in the WHERE clause for (c).*

Task 4: Controlled DELETE with Safety Checks

Medium

2 pts

Write DELETE statements for the following:

- Delete all LEAVE records older than 2 years from today.
- Delete QUALIFICATION records for employees who no longer exist in the EMPLOYEE table.
- Before executing (b), write a SELECT to preview which records would be deleted.

After each DELETE, verify the result with a SELECT COUNT(*).

Task 5: Transaction Management & SAVEPOINT

Hard

3 pts

Demonstrate proper transaction control in the following scenario:

1. Begin a transaction that inserts a new employee and their first payroll record.
2. Set a SAVEPOINT after the employee insert.
3. Insert an intentionally incorrect payroll record (e.g., negative total_amount).
4. ROLLBACK to the savepoint — verify the bad payroll record is gone but the employee remains.
5. Insert the corrected payroll record and COMMIT the full transaction.

Include a SELECT after each step to prove the state of the database at that point.

04 Aggregation Functions

Task 1: Basic Aggregation

Easy

1 pts

Write queries using standard aggregate functions:

- (a) Total number of employees in each department.
- (b) Minimum, maximum, and average salary (amount) across all salary records.
- (c) Total bonus paid out across the entire company.

Task 2: GROUP BY with HAVING

Easy

1 pts

Write GROUP BY queries with HAVING filters:

- (a) List departments where the average employee age exceeds 30.
- (b) Show all job titles where more than 2 employees share that qualification position.
- (c) Find months (from PAYROLL.date) where the total payroll amount exceeds 20,000.

Task 3: Aggregation with Multiple Functions

Medium

2 pts

Write a single query that produces a department summary report. For each department show:

- (a) Department name.
- (b) Total number of employees (COUNT).
- (c) Total payroll amount paid out (SUM of total_amount from PAYROLL).
- (d) Average salary of employees in that department (AVG of SALARY_BONUS.amount).
- (e) Highest salary in the department (MAX) and lowest salary (MIN).

Join the necessary tables and group by department. Order results by total payroll descending.

Task 4: Filtered Aggregation — HAVING with Multiple Conditions

Medium

2 pts

Write queries using GROUP BY and HAVING that combine multiple aggregate conditions:

- (a) List departments where the total payroll (SUM) exceeds 15,000 AND the average salary (AVG) is above 3,000.
- (b) Find qualification positions where more than 2 employees hold that position (COUNT) AND the average age of those employees exceeds 28 (AVG).
- (c) Show all employees who have more than 1 leave record (COUNT of LEAVE rows per emp_ID). Display their full name, department, and leave count.

Task 5: Aggregation Across the Full Schema

Hard

3 pts

Write the following queries that require joining multiple tables before aggregating:

- (a) For each department, show: department name, total employees (COUNT), total bonus paid (SUM of bonus), and the difference between the highest and lowest salary (MAX - MIN). Only include departments with at least 2 employees.
- (b) Find the employee with the highest total payroll amount across all their payroll records (SUM per emp_ID). Show the employee's full name, department, and their total.
- (c) Produce a leave summary: for each department, show the total number of leave records taken by its employees (COUNT), the average number of leave records per employee (AVG), and the department with the most leave records ranked first.

■ For (b) and (c), use a subquery or GROUP BY with ORDER BY — do not use window functions.

05 Joins

Task 1: INNER JOIN — Employee Full Profile

Easy

1 pts

Write a query using INNER JOINS to retrieve a complete employee profile:

Columns: emp_ID, full name (fname || ' ' || lname), department name, job title (from QUALIFICATION), salary amount, latest leave date.

Filter: only employees who have both a payroll record AND a qualification record.

Task 2: LEFT OUTER JOIN — Missing Records

Easy

1 pts

Use LEFT OUTER JOINS to find gaps in data:

- (a) List all employees who have never taken any leave (no matching LEAVE record).
- (b) List all departments that have no salary/bonus records associated with them.

Task 3: Multi-Table JOIN — Payroll Report

Medium

2 pts

Write a single query joining all 6 tables to produce a complete payroll report:

Columns: payroll_ID, employee full name, department, position (from QUALIFICATION), salary amount, bonus, leave reason, total_amount.

Order by department name, then by total_amount descending.

■ You may need multiple LEFT JOINS for LEAVE and QUALIFICATION since not all payroll records may have matching rows.

Task 4: SELF JOIN — Employee Hierarchy

Medium

2 pts

The EMPLOYEE table needs a manager relationship. Add a nullable column:

```
ALTER TABLE EMPLOYEE ADD (manager_id NUMBER REFERENCES EMPLOYEE(emp_ID));
```

Then:

- (a) Update at least 5 employees to have a manager_id pointing to another employee.
- (b) Write a SELF JOIN query that lists each employee alongside their manager's full name.
- (c) List all employees who are themselves managers (appear as manager_id for at least one other employee).

Task 5: CROSS JOIN & Set Operations

Hard

3 pts

(a) Use a CROSS JOIN to generate all possible pairings of employees and departments. Show only pairings where the employee does NOT currently belong to that department. Limit output to 20 rows.

(b) Using set operations:

- UNION: employees who appear in either PAYROLL or LEAVE (or both).
- INTERSECT: employees who appear in both PAYROLL and LEAVE.
- MINUS: employees who appear in PAYROLL but have no LEAVE record.

Explain what each result represents in a business context (comments in SQL).

06 Subqueries

Task 1: Single-Row Subquery

Easy

1 pts

Write queries using single-row subqueries:

- (a) Find all employees whose salary is greater than the average salary of the entire company.
- (b) Retrieve the department with the highest total payroll amount.

Task 2: Multi-Row Subquery with IN / ANY / ALL

Easy

1 pts

- (a) List all employees who work in departments that have at least one salary record with a bonus greater than 500. Use IN.
- (b) Find employees whose salary is greater than ALL salaries in the 'Maintenance' department. Use ALL.
- (c) Find employees whose salary is greater than ANY salary in the 'HR' department. Use ANY.

Task 3: Correlated Subquery

Medium

2 pts

Write correlated subqueries (the inner query references the outer query):

- (a) For each employee, show their name and the number of payroll records they have — without using a JOIN. Use a correlated subquery in the SELECT clause.
- (b) Find all employees who have taken more leave days than the average leave count across all employees. (Hint: count LEAVE rows per employee in the subquery.)

Task 4: EXISTS and NOT EXISTS

Medium

2 pts

- (a) List all departments for which EXISTS at least one payroll record with total_amount > 10,000.
- (b) List all employees for whom NOT EXISTS any qualification record — i.e., employees with no recorded qualification.
- (c) Compare the performance plan of EXISTS vs IN for query (b) by running both and checking the output of EXPLAIN PLAN. Paste the plan output as a SQL comment.

Task 5: Inline Views & WITH Clause (CTE)

Hard

3 pts

- (a) Using an inline view (subquery in FROM clause): find the top 3 highest-paid employees per department.
- (b) Rewrite (a) using a WITH clause (Common Table Expression).
- (c) Chain two CTEs: first CTE calculates department totals; second CTE ranks departments by total. Final query selects the top 2 departments.

07 Views

Task 1: Simple Read-Only View

Easy

1 pts

Create a view named **VW_EMPLOYEE_SUMMARY** that returns:

emp_ID, full name, gender, age, department name, and job title (from QUALIFICATION).

After creating it:

- (a) Query the view to list all female employees over 30.
- (b) Try to INSERT a row through this view and document the Oracle error message you receive.

Task 2: Payroll Dashboard View

Easy

1 pts

Create a view named **VW_PAYROLL_DASHBOARD** that returns for each payroll record:

payroll_ID, employee full name, department, salary amount, bonus, leave reason, payroll date, and total_amount.

Query the view to find the top 5 payroll records by total_amount.

Task 3: Updatable View with CHECK OPTION

Medium

2 pts

Create a view named **VW_ACTIVE_EMPLOYEES** that shows only employees whose age ≥ 18 , with WITH CHECK OPTION.

- (a) Try to INSERT an employee with age = 15 through the view. Document the error.
- (b) Successfully INSERT an employee with age = 25 through the view, then verify with SELECT.
- (c) UPDATE an employee's contact address through the view. Verify the change in the base table.

Task 4: Aggregation View

Medium

2 pts

Create a view named **VW_DEPT_STATS** that shows for each department:

department name, employee count, average salary, total bonus, total payroll amount.

- (a) Query the view to list departments where average salary > 3000 .
- (b) Attempt to DROP the base table JOB_DEPARTMENT while the view exists and document the result.
- (c) Use CREATE OR REPLACE to modify the view to also include max and min salary.

Task 5: Materialized View with Refresh

Hard

3 pts

Create a **Materialized View** named **MVW_MONTHLY_PAYROLL** that pre-computes monthly payroll totals:

Columns: year, month, department_name, total_monthly_payroll, employee_count.

```
CREATE MATERIALIZED VIEW MVW_MONTHLY_PAYROLL
BUILD IMMEDIATE
REFRESH COMPLETE ON DEMAND
AS SELECT ...;
```

- (a) Query the materialized view to compare Q1 vs Q2 totals per department.
- (b) Insert new payroll records, then manually refresh using DBMS_MVIEW.REFRESH.
- (c) Verify the materialized view reflects the new data after refresh.

08 Stored Procedures & Functions

Task 1: Procedure — Add New Employee

Easy

1 pts

Create a stored procedure named **SP_ADD_EMPLOYEE** with IN parameters for all **EMPLOYEE** columns (except **emp_ID** which is auto-generated from a sequence).

The procedure must:

- Validate that **emp_email** is not already in use (raise an exception if duplicate).
- Insert the new employee record.
- Print a confirmation message using **DBMS_OUTPUT.PUT_LINE**.

Test by calling the procedure twice with the same email to confirm the exception fires.

Task 2: Function — Calculate Net Salary

Easy

1 pts

Create a function named **FN_NET_SALARY** that accepts **emp_ID** as input and returns the employee's net salary (amount + bonus) as a **NUMBER**.

Use it in a **SELECT** query to display all employees with their calculated net salary, ordered highest to lowest.

Task 3: Procedure — Process Monthly Payroll

Medium

2 pts

Create a procedure named **SP_PROCESS_PAYROLL** that accepts a department **job_ID** and a pay date.

The procedure must:

- Loop through all employees in that department using a cursor.
- For each employee, compute $\text{total_amount} = \text{salary amount} + \text{bonus} - (\text{leave count} * \text{daily_rate})$.

Assume $\text{daily_rate} = \text{amount} / 30$.

- INSERT a new row into **PAYROLL** for each employee.
- Count how many payroll records were inserted and output the count.

Use an explicit cursor with **OPEN / FETCH / CLOSE** syntax.

Task 4: Procedure — Raise Salary with Audit

Medium

2 pts

Create a table **SALARY_AUDIT**(**audit_id**, **emp_id**, **old_amount**, **new_amount**, **changed_by**, **changed_at**).

Then create a procedure named **SP_RAISE_SALARY**(**p_emp_id**, **p_percentage**) that:

- Retrieves the current salary for the employee.
- Calculates the new salary.
- Updates **SALARY_BONUS.amount**.
- Inserts an audit record into **SALARY_AUDIT**.
- Uses exception handling: if **emp_id** not found, raise **NO_DATA_FOUND** and log the error.

Task 5: Package — Employee Management Package

Hard

3 pts

Create a PL/SQL Package named **PKG_EMPLOYEE_MGMT** containing:

Package Specification (Header):

- A public constant: MAX_LEAVE_DAYS CONSTANT NUMBER := 30;
- A public type: TYPE emp_rec IS RECORD (id NUMBER, fullname VARCHAR2(100), dept VARCHAR2(50));
- Declare all procedure/function signatures: SP_ADD_EMPLOYEE, FN_NET_SALARY, SP_PROCESS_PAYROLL, SP_RAISE_SALARY.

Package Body:

- Implement all four subprograms inside the body.
- Add a private helper function FN_VALIDATE_EMAIL(p_email VARCHAR2) RETURN BOOLEAN used by SP_ADD_EMPLOYEE.

Test by calling PKG_EMPLOYEE_MGMT.SP_ADD_EMPLOYEE(...) and PKG_EMPLOYEE_MGMT.FN_NET_SALARY(1) from an anonymous block.

09 Triggers

Task 1: BEFORE INSERT — Auto-Assign emp_ID

Easy

1 pts

Create a BEFORE INSERT trigger named **TRG_EMP_ID** on the EMPLOYEE table.

If the inserted emp_ID is NULL, the trigger should populate it from a sequence (EMP_SEQ.NEXTVAL).

Test: INSERT an employee without specifying emp_ID and verify it was auto-assigned.

■ This is the Oracle 11g way; on Oracle 12c+ you may also use IDENTITY columns — implement the trigger approach.

Task 2: AFTER INSERT — Welcome Log

Easy

1 pts

Create a table **EMPLOYEE_LOG**(log_id, emp_id, action, log_timestamp).

Create an AFTER INSERT trigger named **TRG_EMP_WELCOME_LOG** on EMPLOYEE that inserts a log row with action = 'NEW HIRE' whenever a new employee is added.

Test by inserting 2 employees and querying EMPLOYEE_LOG.

Task 3: BEFORE UPDATE — Prevent Salary Decrease

Medium

2 pts

Create a BEFORE UPDATE trigger named **TRG_PREVENT_SALARY_CUT** on SALARY_BONUS.

If the new amount is less than the old amount, raise an application error:

```
RAISE_APPLICATION_ERROR(-20001, 'Salary decrease is not allowed.');
```

Test:

(a) Try to update a salary to a lower value — confirm the error fires and the UPDATE is rolled back.

(b) Update a salary to a higher value — confirm it succeeds.

Include a screenshot of both test outputs.

Task 4: AFTER DELETE — Archive Deleted Employees

Medium

2 pts

Create an archive table **EMPLOYEE_ARCHIVE** with the same columns as EMPLOYEE plus (archived_at DATE, archived_by VARCHAR2(50)).

Create an AFTER DELETE trigger named **TRG_ARCHIVE_EMPLOYEE** that, on every DELETE from EMPLOYEE:

- Copies the deleted row into EMPLOYEE_ARCHIVE.
- Sets archived_at = SYSDATE.
- Sets archived_by = USER (the current Oracle schema user).

Test: delete one employee and verify the archive table.

Task 5: Compound Trigger — Payroll Validation & Audit

Hard

3 pts

Create a compound trigger named **TRG_PAYROLL_COMPOUND** on the **PAYROLL** table with the following firing points:

BEFORE STATEMENT: Log the start of a bulk payroll operation to **EMPLOYEE_LOG**.

BEFORE EACH ROW: Validate that `total_amount > 0`. If not, raise an error. Also reject any payroll record for an employee who already has a payroll entry for the same month.

AFTER EACH ROW: Insert an audit record into **SALARY_AUDIT** capturing the new `total_amount`.

AFTER STATEMENT: Count and log the total number of rows affected to **EMPLOYEE_LOG**.

Test with both valid and invalid payroll inserts. Include screenshots.

10 Oracle Scheduler Jobs

Task 1: Simple One-Time Scheduler Job

Easy

1 pts

Create a one-time DBMS_SCHEDULER job named **JOB_GREET_EMPLOYEES** that runs 2 minutes from now and executes an anonymous PL/SQL block that prints 'Payroll System Initialized' to DBMS_OUTPUT and inserts a record into EMPLOYEE_LOG.

```
BEGIN
DBMS_SCHEDULER.CREATE_JOB(
  job_name => 'JOB_GREET_EMPLOYEES',
  job_type => 'PLSQL_BLOCK',
  job_action => 'BEGIN ... END;',
  start_date => SYSTIMESTAMP + INTERVAL '2' MINUTE,
  enabled => TRUE),
);
END;
```

After the job runs, query USER_SCHEDULER_JOB_LOG to confirm execution status.

Task 2: Recurring Job — Daily Leave Report

Easy

1 pts

Create a recurring DBMS_SCHEDULER job named **JOB_DAILY_LEAVE_REPORT** that:

- Runs every day at 07:00 AM.
- Calls the procedure SP_PROCESS_PAYROLL or simply counts leave records taken that day and inserts a summary into EMPLOYEE_LOG.

Use a repeat interval: FREQ=DAILY; BYHOUR=7; BYMINUTE=0; BYSECOND=0.

Show the job definition using: SELECT * FROM USER_SCHEDULER_JOBS WHERE JOB_NAME = 'JOB_DAILY_LEAVE_REPORT';

Task 3: Job with Program & Schedule Objects

Medium

2 pts

Demonstrate separation of concerns using DBMS_SCHEDULER components:

- (a) Create a **Program** named **PROG_MONTHLY_PAYROLL** of type STORED_PROCEDURE pointing to SP_PROCESS_PAYROLL with arguments.
- (b) Create a **Schedule** named **SCH_FIRST_OF_MONTH** with FREQ=MONTHLY; BYMONTHDAY=1; BYHOUR=6.
- (c) Create a **Job** named **JOB_MONTHLY_PAYROLL** that combines the program and schedule.

Query all three objects from USER_SCHEDULER_PROGRAMS, USER_SCHEDULER_SCHEDULES, USER_SCHEDULER_JOBS.

Task 4: Job Error Handling & Notifications

Medium

2 pts

Modify **JOB_DAILY_LEAVE_REPORT** to include error handling:

(a) Wrap the job action in a BEGIN...EXCEPTION...END block that catches any exception and inserts the error message (SQLERRM) into EMPLOYEE_LOG with action = 'JOB_ERROR'.

(b) Set the job's max_failures attribute to 3 so the job disables itself after 3 consecutive failures:

```
DBMS_SCHEDULER.SET_ATTRIBUTE('JOB_DAILY_LEAVE_REPORT', 'max_failures', 3);
```

(c) Manually cause a failure by temporarily making the job action reference a non-existent table, then check USER_SCHEDULER_JOB_LOG for the failure entry.

Task 5: Job Chain — Multi-Step Payroll Pipeline

Hard

3 pts

Create an Oracle Scheduler **Chain** named **CHAIN_PAYROLL_PIPELINE** that implements a full end-of-month payroll pipeline with 3 steps:

Step 1 — STEP_PROCESS: Run SP_PROCESS_PAYROLL for all departments (loop through all job_IDs).

Step 2 — STEP_AUDIT: Only runs if Step 1 succeeded. Inserts a summary row into SALARY_AUDIT with total records processed.

Step 3 — STEP_REPORT: Only runs if Step 2 succeeded. Refreshes MVW_MONTHLY_PAYROLL using DBMS_MVIEW.REFRESH.

Define chain rules so Step 2 only starts on Step 1 SUCCESS, and Step 3 only starts on Step 2 SUCCESS. If any step fails, the chain should stop and log an error.

Create a job named **JOB_PAYROLL_CHAIN** to run this chain on the last day of each month at 11:00 PM.

Submit: full DDL for chain, steps, rules, and job + screenshot of USER_SCHEDULER_CHAIN_STEPS.

Grading Rubric

Criterion	Weight	Details
SQL Correctness	40%	Queries run without errors and return expected results
Logic & Completeness	25%	All sub-tasks addressed; business rules correctly implemented
Code Readability	15%	Meaningful aliases, indentation, SQL comments
Error Handling	10%	Exceptions, SAVEPOINT, rollback logic correctly used
Screenshots / Evidence	10%	Required for Trigger and Scheduler Job tasks

Submission Checklist

- 01_erd_mapping.pdf — ERD diagram (Crow's Foot) + relational schema mapping + DDL
- 02_normalization.sql — 1NF / 2NF / 3NF analysis with decompositions
- 03_dml.sql — All INSERT / UPDATE / DELETE / SELECT tasks
- 04_aggregation.sql — All aggregation queries
- 05_joins.sql — All join queries
- 06_subqueries.sql — All subquery tasks
- 07_views.sql — All view definitions and test queries
- 08_procedures.sql — All stored procedures, functions, and the package
- 09_triggers.sql — All trigger definitions + screenshots
- 10_scheduler.sql — All scheduler job scripts + screenshots

Good luck! — Codeline Bootcamp · Database Fundamentals Track